

Resumen del Proyecto QA - Ceiba Software

Documento de apoyo para sustentación. Este PDF describe la solución de automatización desarrollada en el repositorio **qa-technical-test**.

1. Contexto general

El proyecto automatiza pruebas para:

- UI de **SauceDemo** con Playwright y TypeScript.
- API de **Restful Booker** con Playwright Test.
- Performance para GET /booking con Apache JMeter.

Además se incluye documentación de pruebas y un pipeline CI/CD en GitHub Actions.

2. Estructura de carpetas y archivos

```
.
├── api/
│   └── tests/
│       ├── booking-crud.spec.ts
│       └── booking-negative.spec.ts
├── documentation/
│   ├── DEFECTS.md
│   ├── FEEDBACK.md
│   └── TEST_PLAN.md
├── performance/
│   ├── booking_load_test.jmx
│   ├── README_PERFORMANCE.md
│   └── reports/
├── ui/
│   ├── pages/
│   │   ├── CartPage.ts
│   │   ├── CheckoutPage.ts
│   │   ├── InventoryPage.ts
│   │   └── LoginPage.ts
│   └── tests/
│       └── checkout-negative.spec.ts
```

```
|      |— flujo1-checkout.spec.ts
|      |— flujo2-cart.spec.ts
|      |— login-negative.spec.ts
|— .github/
|   |— workflows/
|       |— ci-cd-pipeline.yml
|— package.json
|— playwright.config.ts
|— tsconfig.json
|— README.md
```

La solución utiliza la arquitectura Page Object Model para separar las interacciones de la UI de los escenarios de prueba.

3. Patrones de pruebas e implementación

3.1 UI

La capa de UI usa **Playwright** con Page Objects:

- **LoginPage**: gestión de login, navegación y validación de URL.
- **InventoryPage**: ordenamiento por precio, selección dinámica de productos y apertura de carrito.
- **CartPage**: validación de cantidades, precios, eliminación de producto y avance a checkout.
- **CheckoutPage**: llenado de formulario, validación de pasos de checkout, cálculo de subtotal/tax/total y confirmación de orden.

Este patrón mejora la mantenibilidad y facilita cambiar selectores sin alterar los tests.

3.2 API

Las pruebas API usan Playwright Test con el cliente HTTP integrado:

- Autenticación con POST /auth.
- Creación de reserva con POST /booking.
- Consulta de reserva con GET /booking/{id}.
- Actualización con PUT /booking/{id}.
- Eliminación con DELETE /booking/{id}.

Se valida el ciclo CRUD completo y se miden tiempos de respuesta para cada petición.

3.3 Performance

Se generó un plan JMeter con dos Thread Groups:

- **Calentamiento:** 1 usuario y 15 segundos.
- **Carga:** 10 usuarios, ramp-up de 30 segundos y duración de 120 segundos.

El objetivo es validar la estabilidad y latencia del endpoint GET /booking bajo carga.

4. Casos de prueba implementados

4.1 UI

Archivo	Descripción	Test	Valida
ui/tests/flujo1-checkout.spec.ts	Compra extremo	Compra extremos y valida total	Selecciona el producto más barato y más caro, valida badge, checkout y total con impuestos.
ui/tests/flujo2-cart.spec.ts	Carrito	Elimina caro y valida checkout	Agrega 3 productos dinámicos, elimina el más caro y verifica el error al continuar sin datos.
ui/tests/checkout-negative.spec.ts	Checkout negativo	Valida campos faltantes	Verifica mensajes de error cuando faltan nombre, apellido y código postal.
ui/tests/login-negative.spec.ts	Login negativo	Muestra error credenciales inválidas	Valida error de login con credenciales inválidas y

Archivo	Descripción	Test	Valida
			permanencia en la página de login.

Total de tests UI: 4 archivos de especificación. Al ejecutarse con los 3 proyectos de navegador (ui, ui-firefox, ui-webkit) se convierten en 12 ejecuciones.

4.2 API

Archivo	Descripción	Test	Valida
api/tests/booking-crud.spec.ts	CRUD de reservas	CRUD completo	Autentica, crea, consulta, actualiza, consulta de nuevo y elimina una reserva. Verifica estado, esquema y tiempos.
api/tests/booking-negative.spec.ts	Reservas negativas	Reserva incompleta falla	Verifica que el servicio rechaza una reserva sin campos obligatorios.
api/tests/booking-negative.spec.ts	Reservas negativas	Reserva inexistente 404	Verifica que consultar una reserva no existente devuelve 404.
api/tests/booking-negative.spec.ts	Reservas negativas	Actualización sin auth rechaza	Verifica que actualizar sin token devuelve 401/403.
api/tests/booking-negative.spec.ts	Reservas negativas	Eliminación sin auth rechaza	Verifica que eliminar sin token devuelve 401/403.
api/tests/booking-negative.spec.ts	Reservas negativas	Reserva cuerpo vacío falla	Verifica que crear con body vacío devuelve error.

Total de tests API: 6 pruebas.

4.3 Performance

Métrica	Resultado de la última ejecución
Muestras	8.491, incluyendo 1 warm-up y 8.490 de carga principal.
Solicitudes exitosas	8.491 de 8.491 (100%).
Errores	0.
Promedio	126 ms.
P90 / P95	155 ms / 179 ms.
Throughput	70,7 solicitudes por segundo.
Máximo	6.149 ms, como valor aislado.

Conclusión: el endpoint respondió sin errores y el P95 fue inferior a 2.000 ms. El máximo aislado recomienda repetir la medición en un ambiente controlado y construir una línea base histórica.

5. Cómo se ejecutan las pruebas

5.1 Comandos principales

- `npm run test:ui`: Ejecuta UI en Chromium, Firefox y WebKit.
- `npm run test:api`: Ejecuta API con Playwright Test.
- `npm run test:performance`: Ejecuta JMeter.
- `npm run test:all`: Ejecuta API y UI.
- `npm run test:ci`: Ejecuta API, UI y performance.

5.2 Ejecución UI con exploración visual

- `npm run test:ui:headed`: muestra los navegadores.
- `npm run test:ui:headed:chromium`: ejecuta solo Chromium visible.
- `npm run test:ui:debug`: abre el modo debug de Playwright.

6. Validación de resultados y evidencia

Los resultados quedan disponibles en:

- playwright-report/: reporte HTML de Playwright.
- playwright-results/: artefactos generados por Playwright.
- performance/reports/html-report/: reporte HTML de JMeter.

Para abrir el reporte Playwright:

```
npm run report
```

Para abrir el reporte JMeter:

```
Start-Process .performance  
portshtml-reportindex.html
```

En Windows, `npm run test:performance` utiliza `scripts/run-performance.js`, que ejecuta JMeter mediante Java y limpia los resultados anteriores antes de iniciar una nueva medición.

7. Configuración relevante

Archivo principal de configuración:

```
playwright.config.ts
```

Puntos clave:

- `baseUrl` para UI: `https://www.saucedemo.com`.
- `baseUrl` para API: `https://restful-booker.herokuapp.com`.
- `trace`, `screenshot`, `video` se retienen solo en fallos.
- `outputDir` configurado en `playwright-results`.

Archivo TypeScript:

```
tsconfig.json
```

Puntos clave:

- `target`: ES2020

- `module: CommonJS`
- `moduleResolution: node`
- `strict: true`
- `skipLibCheck: true`

8. Asuntos importantes de sustentación

- El proyecto está diseñado para pruebas E2E en UI, pruebas de contrato API y prueba de carga.
- Se emplea Page Object Model en UI para mantener separada la lógica de interacción del flujo de pruebas.
- Los nombres de los tests UI se estandarizaron a español para una presentación más clara.
- Las pruebas API validan tanto el flujo correcto como escenarios negativos de seguridad/autenticación.
- Los resultados se validan en reportes HTML, que son evidencia de ejecución y fallos.